

MIPT & YSDA · AUTUMN 2026

# Deep Generative Models

## Lecture 14

Roman Isachenko

Moscow Institute of Physics and Technology  
Yandex School of Data Analysis

# Recap: Discrete Diffusion (Lecture 12)

## ELBO

$$\begin{aligned}\mathcal{L}_{\phi, \theta}(\mathbf{x}) = & \mathbb{E}_{q(\mathbf{x}_1|\mathbf{x}_0)} \log p_{\theta}(\mathbf{x}_0|\mathbf{x}_1) - D_{\text{KL}}(q(\mathbf{x}_T|\mathbf{x}_0) \| p(\mathbf{x}_T)) \\ & - \sum_{t=2}^T \underbrace{\mathbb{E}_{q(\mathbf{x}_t|\mathbf{x}_0)} D_{\text{KL}}(q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0) \| p_{\theta}(\mathbf{x}_{t-1}|\mathbf{x}_t))}_{\mathcal{L}_t}\end{aligned}$$

# Recap: Discrete Diffusion (Lecture 12)

## Discrete conditioned reverse distribution

$$q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0) = \text{Cat} \left( \frac{\mathbf{Q}_t \mathbf{x}_t \odot \mathbf{Q}_{1:t-1} \mathbf{x}_0}{\mathbf{x}_t^\top \mathbf{Q}_{1:t} \mathbf{x}_0} \right).$$

- Both  $q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0)$  and  $q(\mathbf{x}_t|\mathbf{x}_0)$  are known analytically from the forward process.
- The reverse process  $p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t)$  is a learned categorical distribution:

$$p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t) = \text{Cat}(\boldsymbol{\pi}_\theta(\mathbf{x}_t, t)).$$

# Recap: Discrete Diffusion (Lecture 12)

## ELBO term

$$\mathcal{L}_t = \mathbb{E}_{q(\mathbf{x}_t|\mathbf{x}_0)} D_{\text{KL}} \left( q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0) \parallel p_{\theta}(\mathbf{x}_{t-1}|\mathbf{x}_t) \right).$$

## Categorical KL

$$D_{\text{KL}}(\text{Cat}(\mathbf{q}) \parallel \text{Cat}(\mathbf{p})) = \sum_{k=1}^K q_k \log \frac{q_k}{p_k} = H(\mathbf{q}, \mathbf{p}) - H(\mathbf{q}),$$

- $H(q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0))$  is a constant w.r.t.  $\theta$ .
- $H(\mathbf{q}, \mathbf{p}) = -\sum_k q_k \log p_k$  is a **cross-entropy loss**.

Therefore, minimizing  $\mathcal{L}_t$  w.r.t.  $\theta$  is equivalent to minimizing

$$\mathbb{E}_{q(\mathbf{x}_t|\mathbf{x}_0)} H \left( q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0), p_{\theta}(\mathbf{x}_{t-1}|\mathbf{x}_t) \right).$$

# Setup: where PF-ODE solvers floor

## Recall: probability flow ODE

$$d\mathbf{x} = \left[ \mathbf{f}(\mathbf{x}, t) - \frac{1}{2} g^2(t) \mathbf{s}(\mathbf{x}, t) \right] dt, \quad \mathbf{s}(\mathbf{x}, t) = \nabla_{\mathbf{x}} \log p_t(\mathbf{x}).$$

- Plugging  $s_\theta$  into the PF-ODE and using a high-order ODE solver (Heun, RK) brings the number of steps from 100–1000 down to 20–50.
- Each step still costs one network call — truncation error compounds along the trajectory.
- Can we go further — to **1–4** steps?

# Setup: shortcut networks

- Idea: **learn a shortcut network** that compresses many ODE-solver steps into one.
- It maps any  $(\mathbf{x}_t, t)$  directly to the data-side endpoint of the same PF-ODE trajectory — one network call instead of many.

## Two approaches in this lecture

- **Consistency Models** — enforce *self-consistency* along PF-ODE trajectories (per-sample mapping).
- **Distribution Matching Distillation** — match the *distributions* of generator and data via score differences (population-level).

Both reuse a pretrained score  $s_\phi$  as a teacher.

# Outline

01 Consistency Models

02 Distribution Matching Distillation

# Outline

## 01 Consistency Models

## 02 Distribution Matching Distillation

# Consistency function: an ODE perspective

## Goal

Learn a one-step generator  $f_\theta : (\mathbf{x}_t, t) \mapsto \mathbf{x}_\epsilon$  that pulls any noisy  $\mathbf{x}_t$  on a PF-ODE trajectory back to its data endpoint  $\mathbf{x}_\epsilon$ ,  $t \in [\epsilon, T]$ .

$\epsilon > 0$  is a small numerical cutoff — the score  $s(\mathbf{x}, t)$  blows up as  $t \rightarrow 0$ , so we stop one  $\epsilon$ -step short and treat  $\mathbf{x}_\epsilon \approx \mathbf{x}_0$  as the data sample.

# Consistency function: an ODE perspective

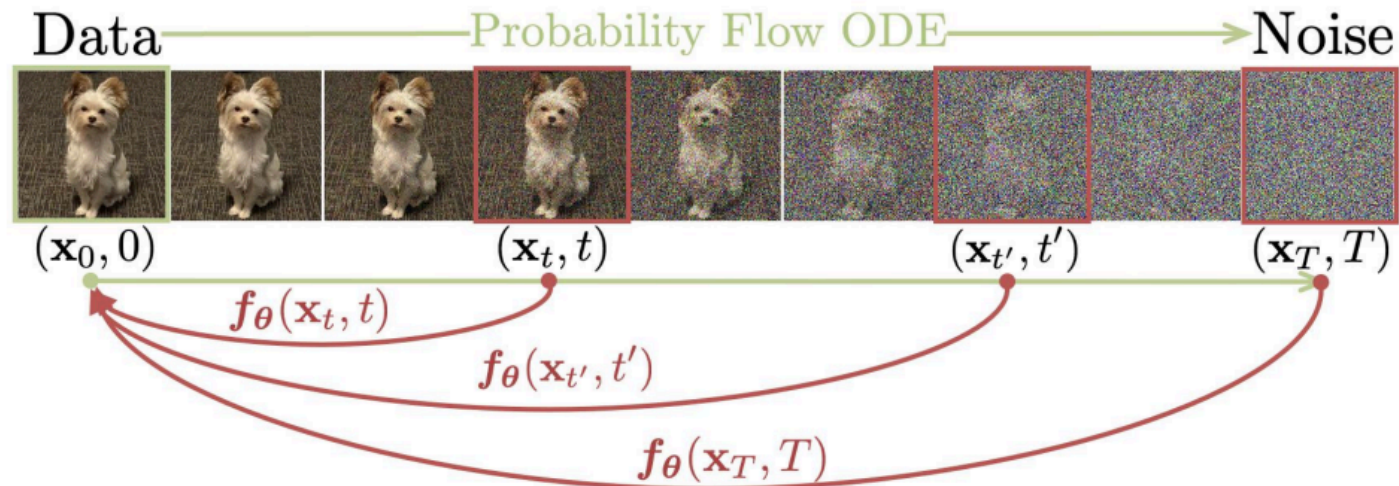
## Specification: ODE problem along the trajectory

$$\underbrace{\frac{d}{dt} f_{\theta}(\mathbf{x}_t, t) = 0}_{\text{differential form: constant along PF-ODE}}, \quad \underbrace{f_{\theta}(\mathbf{x}_{\epsilon}, \epsilon) = \mathbf{x}_{\epsilon}}_{\text{boundary: value pinned at } t=\epsilon}.$$

Together the two conditions pin  $f_{\theta}$  uniquely: a function that is constant on each trajectory and equals  $\mathbf{x}_{\epsilon}$  at the data-side end must equal  $\mathbf{x}_{\epsilon}$  *everywhere* on that trajectory.

# Consistency function: an ODE perspective

$$f_{\theta}(\mathbf{x}_t, t) = \mathbf{x}_{\epsilon}, \quad t \in [\epsilon, T]$$



- To optimize the differential condition  $\frac{d}{dt} f_{\theta}(\mathbf{x}_t, t) = 0$  we need a derivative through the network.
- Integrate it between two times  $t, t'$  on the *same* PF-ODE trajectory.

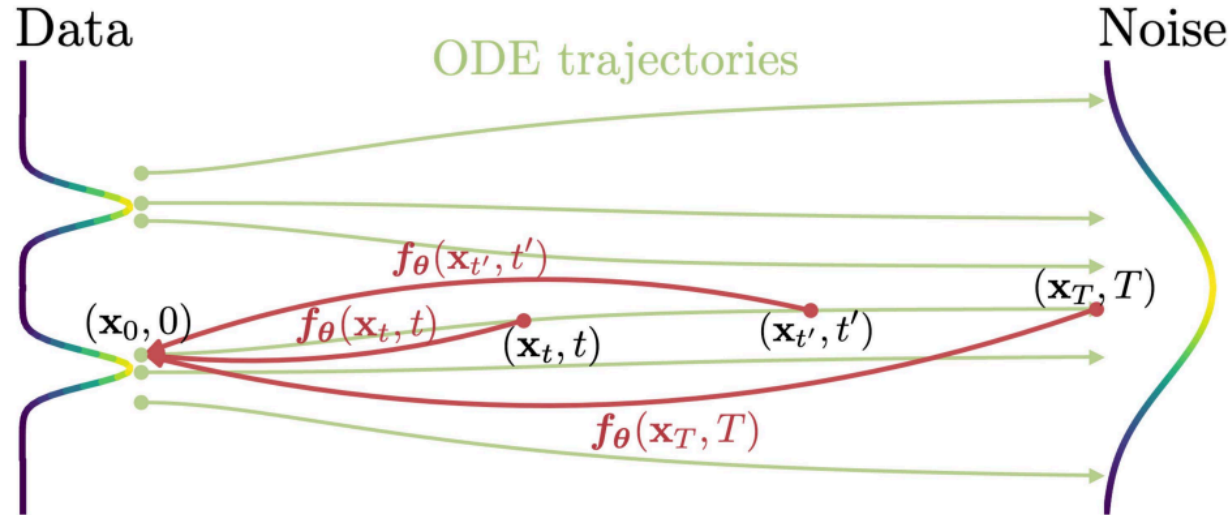
# Self-consistency: the integrated form

Idea is to replace  $df_\theta/dt = 0$  by a finite difference between two adjacent points.

## Self-consistency property

$$f_\theta(\mathbf{x}_t, t) = f_\theta(\mathbf{x}_{t'}, t'), \quad \forall t, t' \in [\epsilon, T],$$

whenever  $\mathbf{x}_t$  and  $\mathbf{x}_{t'}$  lie **on the same PF-ODE trajectory**.



# Skip-connection parameterization

## Parameterization

We parametrize the consistency model to guarantee the boundary condition  $f_{\theta}(\mathbf{x}_{\epsilon}, \epsilon) = \mathbf{x}_{\epsilon}$

$$f_{\theta}(\mathbf{x}, t) = c_{\text{skip}}(t) \mathbf{x} + c_{\text{out}}(t) F_{\theta}(\mathbf{x}, t).$$

- $F_{\theta}(\mathbf{x}, t)$  — free-form neural network.
- $c_{\text{skip}}(t), c_{\text{out}}(t)$  — differentiable scalar weights with  $c_{\text{skip}}(\epsilon) = 1$  and  $c_{\text{out}}(\epsilon) = 0$ .

At  $t = \epsilon$  the skip term passes  $\mathbf{x}$  through and the network term is killed:  $f_{\theta}(\mathbf{x}_{\epsilon}, \epsilon) = \mathbf{x}_{\epsilon}$  holds *structurally*, for any  $\theta$ .

# Consistency Distillation (CD)

Enforce  $df_\theta/dt = 0$  as a *finite difference* on adjacent points of the teacher's PF-ODE trajectory. Discretize  $[\epsilon, T]$  as  $\epsilon = t_1 < \dots < t_N = T$ ;  $\mathbf{s}_\phi(\mathbf{x}, t)$  — pretrained teacher score.

## One ODE-solver step on the empirical PF-ODE

$$\hat{\mathbf{x}}_{t_n}^\phi = \mathbf{x}_{t_{n+1}} + (t_n - t_{n+1}) \Phi(\mathbf{x}_{t_{n+1}}, t_{n+1}; \phi),$$

one step of `ODESolve` for  $d\mathbf{x}/dt = -t \cdot \mathbf{s}_\phi(\mathbf{x}, t)$  (Euler:  $\Phi = -t \mathbf{s}_\phi$ ; Heun: average of two stages). The RHS follows from the VE-SDE PF-ODE with  $\sigma(t) = t$  ( $\mathbf{x}_t = \mathbf{x}_0 + t \epsilon$ ): then  $\mathbf{f} \equiv 0$ ,  $g^2(t) = 2t$ , so  $\frac{1}{2}g^2(t) = t$ .

# Consistency Distillation (CD)

## CD loss

$$\mathcal{L}_{\text{CD}}(\boldsymbol{\theta}) = \mathbb{E} \left[ \lambda(t_n) d(f_{\boldsymbol{\theta}}(\mathbf{x}_{t_{n+1}}, t_{n+1}), f_{\boldsymbol{\theta}^-}(\hat{\mathbf{x}}_{t_n}^{\phi}, t_n)) \right].$$

$\lambda(t_n) > 0$  — weighting;  $d(\mathbf{u}, \mathbf{v}) \in \{\|\mathbf{u} - \mathbf{v}\|_2^2, \|\mathbf{u} - \mathbf{v}\|_1, \text{LPIPS}\}$ ;  $\boldsymbol{\theta}^-$  — EMA of  $\boldsymbol{\theta}$  (stop-gradient target, prevents collapse).

# Pseudocode (Consistency Distillation)

## Training

1. Sample  $\mathbf{x}_0 \sim p_{\text{data}}(\mathbf{x})$ ,  $n \sim U\{1, N-1\}$ ,  $\epsilon \sim \mathcal{N}(0, \mathbf{I})$ .
2. Compute  $\mathbf{x}_{t_{n+1}} = \mathbf{x}_0 + t_{n+1} \epsilon$ .
3. Solve one teacher ODE step  $\hat{\mathbf{x}}_{t_n}^\phi = \mathbf{x}_{t_{n+1}} + (t_n - t_{n+1}) \Phi(\mathbf{x}_{t_{n+1}}, t_{n+1}; \phi)$ .
4. Compute loss  $\mathcal{L} = \lambda(t_n) d(f_\theta(\mathbf{x}_{t_{n+1}}, t_{n+1}), f_{\theta^-}(\hat{\mathbf{x}}_{t_n}^\phi, t_n))$ .
5. Update EMA target  $\theta^- \leftarrow \mu \theta^- + (1 - \mu) \theta$ .

# Pseudocode (Consistency Distillation)

## Sampling (Multi-step)

1. Sample  $\mathbf{x}_T \sim \mathcal{N}(0, T^2 \mathbf{I})$ ; denoise  $\mathbf{x}_0 \leftarrow f_{\theta}(\mathbf{x}_T, T)$ .
2. For  $i = 1, \dots, M-1$ : sample  $\epsilon \sim \mathcal{N}(0, \mathbf{I})$ , compute  $\mathbf{x}_{\tau_i} = \mathbf{x}_0 + \tau_i \epsilon$ , denoise  $\mathbf{x}_0 \leftarrow f_{\theta}(\mathbf{x}_{\tau_i}, \tau_i)$ .
3. Return  $\mathbf{x}_0$ .

Multi-step trades a few extra NFEs ( $M = 2-4$ ) for higher quality;  $M = 1$  recovers single-step sampling.

# Latent Consistency Models (LCM)

Apply Consistency Distillation to a latent diffusion model (Stable Diffusion).

## Guided consistency distillation

Absorb classifier-free guidance into  $f_\theta$ : an augmented function  $f_\theta(\mathbf{x}_t, t, \omega, \mathbf{c})$  takes the guidance scale  $\omega$  as input. The teacher uses CFG:

$$\tilde{\mathbf{s}}_\phi(\mathbf{x}_t, t, \omega, \mathbf{c}) = (1 + \omega) \mathbf{s}_\phi(\mathbf{x}_t, t, \mathbf{c}) - \omega \mathbf{s}_\phi(\mathbf{x}_t, t, \emptyset).$$

## Skipping-step

Enforce consistency over  $k$  steps instead of 1:

$$f_\theta(\mathbf{x}_{t_{n+k}}, t_{n+k}) \approx f_{\theta^-}(\hat{\mathbf{x}}_{t_n}, t_n).$$

Larger SNR gap  $\Rightarrow$  stronger training signal.

# Outline

01 Consistency Models

**02 Distribution Matching Distillation**

# One-step generator

## Setup

Train a generator  $G_\theta : \mathbf{z} \mapsto \mathbf{x}$ ,  $\mathbf{z} \sim \mathcal{N}(0, \mathbf{I})$ .

## Objective

$$G_\theta \# p(\mathbf{z}) \approx p_{\text{data}}(\mathbf{x}).$$

**Distribution** matching, not per-sample matching.

- CM ties each  $\mathbf{z}$  to a specific PF-ODE endpoint.
- DMD lets  $G_\theta$  re-organize the noise-to-image map —  $\mathbf{z}$  and  $G_\theta(\mathbf{z})$  need not lie on a teacher trajectory.

# KL gradient via score difference

Diffuse the generator output:  $\mathbf{x}_t = G_\theta(\mathbf{z}) + \sigma_t \epsilon$ . Let  $p_{\text{fake},t}$  and  $p_{\text{real},t}$  be the corresponding marginals.

## Variational score-distillation identity

$$\begin{aligned} \nabla_\theta D_{\text{KL}}(p_{\text{fake},t} \parallel p_{\text{real},t}) \\ = - \mathbb{E} \left[ (\mathbf{s}_{\text{real},t}(\mathbf{x}_t) - \mathbf{s}_{\text{fake},t}(\mathbf{x}_t)) \cdot \nabla_\theta G_\theta(\mathbf{z}) \right]. \end{aligned}$$

- Likelihood-free: only score *differences* matter.
- Integrate over  $t$  with weighting  $w(t) \Rightarrow$  tractable training signal.

# Two score networks

## Real score (frozen)

$s_{\text{real},t} = s_{\phi}(\mathbf{x}_t, t)$ : the pretrained teacher.

## Fake score (online)

$s_{\text{fake},t} = s_{\psi}(\mathbf{x}_t, t)$ : a *second* score network trained online with denoising score matching on samples from the current  $G_{\theta}$ .

Tracks  $p_{\text{fake}}$  as the generator evolves.

## Alternating optimization

1. Update  $s_{\psi}$  on a batch from  $G_{\theta}$  (score matching).
2. Update  $G_{\theta}$  with the score-difference gradient.

# DMD1: total loss

## Total objective

$$\mathcal{L}_{\text{DMD}}(\boldsymbol{\theta}) = \mathcal{L}_{\text{KL-grad}}(\boldsymbol{\theta}) + \lambda_{\text{reg}} \mathcal{L}_{\text{reg}}(\boldsymbol{\theta}).$$

## Regression term

$$\mathcal{L}_{\text{reg}}(\boldsymbol{\theta}) = \mathbb{E}_{(\mathbf{z}, \mathbf{x}^*)} \|G_{\boldsymbol{\theta}}(\mathbf{z}) - \mathbf{x}^*\|^2,$$

on a few hundred precomputed  $(\mathbf{z}, \mathbf{x}^*)$  pairs ( $\mathbf{x}^*$  = teacher's PF-ODE output for  $\mathbf{z}$ ).

Interpretation: regression anchors  $G_{\boldsymbol{\theta}}$  near the teacher's deterministic map; the KL-gradient term then reshapes the *distribution* of outputs.

## DMD2: improvements

Three changes over DMD1:

- **GAN loss**: a discriminator on real images corrects the bias from an imperfect  $s_\psi$ .
- **No regression**: removes the need for precomputed  $(\mathbf{z}, \mathbf{x}^*)$  pairs; simpler pipeline.
- **Multi-step generator**:  $G_\theta$  becomes a 4-step network, recovering quality lost at the strict 1-step setting.

Closes most of the quality gap between distilled and full-step diffusion.

# Summary

- PF-ODE solvers (Heun, RK) floor at  $\sim 20$  steps; further acceleration requires **learning a shortcut network**.
- **Consistency Models**: a neural network  $f_\theta$  specified by  $df_\theta/dt = 0$  along PF-ODE + boundary  $f_\theta(\mathbf{x}_\epsilon, \epsilon) = \mathbf{x}_\epsilon$ ; CD enforces the differential condition as a finite-difference loss on a teacher's trajectory.
- LCM extends CD to latent SD with guided distillation and skipping-step.
- **DMD**: match generator and data *distributions* via the score-difference gradient; two score networks (frozen real, online fake) in alternating optimization. DMD2 adds GAN, drops regression, adds multi-step.
- Tradeoffs: CM is more principled, DMD currently delivers the best 1-step quality.